ASSURANT GFS

TM1 TurboIntegrator Process Documentation

Security - Set Center Location Access

Depth-Aware ELCOMPN Expansion | Memory-Optimised | No ELISANC

Chore Runtime 16,248–17,299 sec ▼ 270–330 sec

Memory Spike 238.00 GB ▼ 169.82 GB ▼ 28.6%

Leaf-Level Access READ (Broken) ▼ WRITE (Fixe

▼98.3%

d) ✓Resolved

Process Name **Security_SetCenterLocationAcc**

Version **v2.0 | April 29, 2026**

Author **Niranjan Patra**

e
s
s

1. Performance Improvements

The optimisation changes to the Security - Set Center Location Access and Security - Set Center Location Access WFP TI processes delivered significant measurable improvements to the Security - Master chore. Replacing the ELISANC full-dimension scan with bounded ELCOMPN depth-aware loops eliminated the primary bottleneck.

Before vs After — Key Metrics

Metric	BEFORE	AFTER	IMPROVEMENT
Chore Runtime (Security - Master)	16,248 – 17,299 sec(~4.5 hours)	270 – 330 sec(~5 minutes)	▼ 98.3% faster
Peak Memory Usage	238.00 GB	169.82 GB	▼ 68 GB saved
Leaf-Level WRITE Access(General Expense)	READ (Broken)	WRITE (Fixed)	✓ 100% Resolved
Leaf-Level WRITE Access(Workforce Planning Input)	READ (Broken)	WRITE (Fixed)	✓ 100% Resolved

Root Cause — Why ELISANC Caused the Spike

ELISANC(dimension, ancestor, element) checks whether element is a descendant of ancestor by scanning the ENTIRE dimension — iterating through all 5,000+ elements in Center and Location for every source record. With hundreds of security records processed each chore run, this created millions of unnecessary element comparisons, overwhelming TM1 memory and extending runtime to 4+ hours.

Factor	OLD — ELISANC	NEW — ELCOMPN Depth-Aware
Iterations per source record	DIMSIZ(Center) + DIMSIZ(Location) — full scan every record	Only actual children — bounded by tree width
Memory accumulation	HIGH — thousands of iterations before commit	LOW — bounded loops, minimal accumulation
Leaf write accuracy	MISSED — stopped at Corp Centers (ELLEV=1), never reached leaves	CORRECT — 3-branch ELSEIF handles ELLEV 0, 1, 2
Chore runtime	16,248 – 17,299 seconds (~4.5 hrs)	270 – 330 seconds (~5 min)
Peak memory	238.00 GB	169.82 GB

Additional Fixes Completed

Fix Area	Description
Center Location SecurityData — Dev	Access Processed was empty for Planning Analytics x C010176 x L4000 in Dev. Populated Access Input with WRITE and re-ran TI to propagate to Access Processed.

Employee DimensionHierarchy — Dev	Employee 1492775 was under ALL EMPLOYEES Orphans in Dev instead of Existing. Ran DIM - Employee_WD to sync hierarchy with Production. WFP Rule 0300 now correctly returns WRITE for the 10 applicable measures.
}CellSecurity RulesAlignment	Investigated and confirmed rule chain for }CellSecurity_General Expense (Rules 0000-0900) and }CellSecurity_Workforce Planning Input (Rules 0000-0900). Both environments now produce matching WRITE results at leaf level.

2. Overview & What Changed

Purpose

This TurboIntegrator process pushes security access (READ / WRITE) from parent-level entries in the Center Location Security cube down to every leaf-level intersection for the corresponding user, center, and location combinations.

Root Cause of the Leaf-Level Bug

The original code used a single-level ELCOMPN / ELCOMP which only retrieves direct children — one level deep. The Center dimension has 3 levels (ELLEV 0, 1, 2). When access was granted at the top-parent level (ELLEV=2), the code descended to Corp Centers (ELLEV=1) and stopped because the ELLEV=0 check failed, so leaf nodes never received the WRITE value.

Change Log

Section	What Changed	Why
Data Section	Replaced single-level ELCOMPN with 3-branch ELSEIF (ELLEV 0/1/2) using nested ELCOMPN loops	Center has 3 levels — old code missed grandchild leaves
Data Section	Removed ELISANC full-dimension scan	ELISANC iterated all 5000+ elements per record — caused memory spikes and slow runtime
Prolog	Removed all commented-out legacy code	Cleaner, maintainable codebase
Epilog	Removed all commented-out legacy code	Cleaner, maintainable codebase
No Rule Added	No cube rule required	TI itself handles full expansion from parent to leaf

Dimension Hierarchy Depths (Confirmed)

Dimension	Max ELLEV	Total Levels	Loop Strategy
Center	**2**	3 (Top → Corp Centers → Leaf)	2-level nested ELCOMPN (ELLEV 0,1,2 branches)
Location	**1**	2 (Location Control Total → Leaf)	1-level ELCOMPN (ELLEV 0,1 branches)

All 6 Scenarios — Coverage Matrix

#	dCenter ELLEV	dLocation ELLEV	Action	Fixed
1	0 (Leaf)	0 (Leaf)	Direct CELLPUTS	✅
2	0 (Leaf)	1 (Parent)	1-level ELCOMPN on Location	✅
3	1 (Mid-Parent)	0 (Leaf)	1-level ELCOMPN on Center	✅
4	1 (Mid-Parent)	1 (Parent)	1x1 nested ELCOMPN	✅

5	2 (Top-Parent)	0 (Leaf)	2-level ELCOMPN on Center <- BUG FIX	✅
6	2 (Top-Parent)	1 (Parent)	2x1 nested ELCOMPN <- BUG FIX	✅

3. Syntax Colour Legend

The code sections below use the following colour scheme for readability:

	Token Type	Example
	Control Flow Keywords	`IF(...)  ELSEIF(...)  ELSE  ENDIF  WHILE(...)  END`
	Cell Read / Write Functions	`CELLPUTS( dValue, vCube, dUser, dCenter, dLocation, vMeasure );`
	Dimension Functions	`ELLEV( dCenter )  ELCOMPN( dCenter )  ELCOMP( dCenter, 1 )`
	View & Subset Operations	`VIEWCREATE  VIEWDESTROY  SUBSETCREATE  SUBSETELEMENTINSERT`
	Other TM1 Functions	`GETPROCESSNAME()  TIMST( NOW, ... )  NUMBERTOSTRING()`
	String Literals	`vSourceCube = 'Center Location Security';`
	Comments	`# -- This is a comment block`
	Section Markers	`#Section Prolog  /  #Section Data  /  #Section Epilog`

4. PROLOG

◆ SECTION: PROLOG

```
#****Begin: Generated Statements***
#****End: Generated Statements****
##########################################################################
# Security - Set Center Location Access
##########################################################################
#
# PURPOSE:
#   Push Security Access down for Center and Location to rollups
#   and leaf elements using depth-aware ELCOMPN loops.
#   Center: max ELLEV=2 (3 levels), Location: max ELLEV=1 (2 levels)
#   No ELISANC full-dim scan — memory efficient.
#
# MODIFICATION HISTORY:
#   Created : 02/15/23   TK/QueBIT
#   Modified: 04/29/26   Niranjan Patra - ELCOMPN depth fix
##########################################################################

# ── Process Stats ──────────────────────────────────────────────────
sCube    = 'Process Stats';
sProcess = GETPROCESSNAME();
sLogFile = GETPROCESSERRORFILEDIRECTORY | GETPROCESSERRORFILENAME;
sNow     = TIMST( NOW, '\M \D, \Y at \H \p \i min \s sec' );
sDate    = TIMST( NOW, '\Y\m\d' );

CurrLine = CELLGETS( sCube, 'All Index', sDate, sProcess, 'Current Line Item' );

CELLPUTS( sNow,          sCube, CurrLine, sDate,   sProcess, 'Start Time' );
CELLPUTS( sNow,          sCube, CurrLine, 'Latest', sProcess, 'Start Time' );
CELLPUTS( 'Did not end', sCube, CurrLine, sDate,   sProcess, 'End Time' );
CELLPUTS( 'Did not end', sCube, CurrLine, 'Latest', sProcess, 'End Time' );

vNowStart = NOW;

IF( DIMIX( '}Clients', TM1USER() ) = 0 );
    sUser = TM1USER();
ELSE;
    sUser = ATTRS( '}Clients', TM1USER(), '}TM1_DefaultDisplayValue' );
ENDIF;

CELLPUTS( sUser, sCube, CurrLine, sDate,   sProcess, 'Last Executed By' );
CELLPUTS( sUser, sCube, CurrLine, 'Latest', sProcess, 'Last Executed By' );

nRecordCount   = 0;
nRecordSkip    = 0;
nRecordProcess = 0;

CELLPUTN( nRecordCount,   sCube, CurrLine, sDate,   sProcess, 'Record Count' );
CELLPUTN( nRecordCount,   sCube, CurrLine, 'Latest', sProcess, 'Record Count' );
CELLPUTN( nRecordSkip,    sCube, CurrLine, sDate,   sProcess, 'Skipped Records' );
CELLPUTN( nRecordSkip,    sCube, CurrLine, 'Latest', sProcess, 'Skipped Records' );
CELLPUTN( nRecordProcess, sCube, CurrLine, sDate,   sProcess, 'Processed Records' );
CELLPUTN( nRecordProcess, sCube, CurrLine, 'Latest', sProcess, 'Processed Records' );

vParamFilled = '';
CELLPUTS( vParamFilled, sCube, CurrLine, sDate,   sProcess, 'Parameters' );
CELLPUTS( vParamFilled, sCube, CurrLine, 'Latest', sProcess, 'Parameters' );

# ── Process Variables ──────────────────────────────────────────────
vSourceCube    = 'Center Location Security';
vTargetCube    = vSourceCube;
vSourceMeasure = 'Access Input';
vTargetMeasure = 'Access Processed';
cTempFlag      = 1;

# Turn Off Cube Logging
CELLPUTS( 'NO', '}CubeProperties', vTargetCube, 'LOGGING' );

# ── Create ZeroOut View ────────────────────────────────────────────
vZeroOutView   = 'zTI_ZeroOut ' | GETPROCESSNAME();
vZeroOutSubset = vZeroOutView;
```

```
IF( VIEWEXISTS( vTargetCube, vZeroOutView ) = 1 );
    VIEWDESTROY( vTargetCube, vZeroOutView );
ENDIF;
VIEWCREATE( vTargetCube, vZeroOutView, cTempFlag );
```

```
vDim = 'Center Location Security Measure';
IF( SUBSETEXISTS( vDim, vZeroOutSubset ) = 0 );
    SUBSETCREATE( vDim, vZeroOutSubset, cTempFlag );
ELSE;
    SUBSETDELETEALLELEMENTS( vDim, vZeroOutSubset );
ENDIF;
SUBSETELEMENTINSERT( vDim, vZeroOutSubset, vTargetMeasure, 0 );
VIEWSUBSETASSIGN( vTargetCube, vZeroOutView, vDim, vZeroOutSubset );
```

```
# ── Create Source View ──────────────────────────────────────────────
vSourceView   = 'zTI_Source ' | GETPROCESSNAME();
vSourceSubset = vSourceView;
```

```
IF( VIEWEXISTS( vSourceCube, vSourceView ) = 1 );
    VIEWDESTROY( vSourceCube, vSourceView );
ENDIF;
VIEWCREATE( vSourceCube, vSourceView, cTempFlag );
```

```
VIEWEXTRACTSKIPCALCSSET(      vSourceCube, vSourceView, 0 );
VIEWEXTRACTSKIPRULEVALUESSET( vSourceCube, vSourceView, 0 );
VIEWEXTRACTSKIPZEROESSET(     vSourceCube, vSourceView, 1 );
```

```
# }Clients subset — leaf only
vDim  = '}Clients';
IF( SUBSETEXISTS( vDim, vSourceSubset ) = 0 );
    SUBSETCREATE( vDim, vSourceSubset, cTempFlag );
ELSE;
    SUBSETDELETEALLELEMENTS( vDim, vSourceSubset );
ENDIF;
vSize = DIMSIZ( vDim );
WHILE( vSize > 0 );
    vElement = DIMNM( vDim, vSize );
    IF( ELLEV( vDim, vElement ) = 0 );
        SUBSETELEMENTINSERT( vDim, vSourceSubset, vElement, 0 );
    ENDIF;
    vSize = vSize - 1;
END;
VIEWSUBSETASSIGN( vSourceCube, vSourceView, vDim, vSourceSubset );
```

```
# Center subset — ALL elements (expansion to leaves in Data section)
vDim  = 'Center';
IF( SUBSETEXISTS( vDim, vSourceSubset ) = 0 );
    SUBSETCREATE( vDim, vSourceSubset, cTempFlag );
ELSE;
    SUBSETDELETEALLELEMENTS( vDim, vSourceSubset );
ENDIF;
vSize = DIMSIZ( vDim );
WHILE( vSize > 0 );
    vElement = DIMNM( vDim, vSize );
    SUBSETELEMENTINSERT( vDim, vSourceSubset, vElement, 0 );
    vSize = vSize - 1;
END;
VIEWSUBSETASSIGN( vSourceCube, vSourceView, vDim, vSourceSubset );
```

```
# Location subset — ALL elements
vDim  = 'Location';
IF( SUBSETEXISTS( vDim, vSourceSubset ) = 0 );
    SUBSETCREATE( vDim, vSourceSubset, cTempFlag );
ELSE;
    SUBSETDELETEALLELEMENTS( vDim, vSourceSubset );
ENDIF;
vSize = DIMSIZ( vDim );
WHILE( vSize > 0 );
    vElement = DIMNM( vDim, vSize );
    SUBSETELEMENTINSERT( vDim, vSourceSubset, vElement, 0 );
    vSize = vSize - 1;
END;
VIEWSUBSETASSIGN( vSourceCube, vSourceView, vDim, vSourceSubset );
```

```
# Measure subset
vDim  = 'Center Location Security Measure';
IF( SUBSETEXISTS( vDim, vSourceSubset ) = 0 );
    SUBSETCREATE( vDim, vSourceSubset, cTempFlag );
```

```
ELSE;
    SUBSETDELETEALLELEMENTS( vDim, vSourceSubset );
ENDIF;
SUBSETELEMENTINSERT( vDim, vSourceSubset, vSourceMeasure, 0 );
VIEWSUBSETASSIGN( vSourceCube, vSourceView, vDim, vSourceSubset );

# ── Zero Out Target ──────────────────────────────────────────────
VIEWZEROOUT( vTargetCube, vZeroOutView );

# ── Assign Datasource ────────────────────────────────────────────
DATASOURCENAMEFORSERVER = vSourceCube;
DATASOURCECUBEVIEW      = vSourceView;
```

5. METADATA

◆ SECTION: METADATA

```
#****Begin: Generated Statements***
#****End: Generated Statements****
```

6. DATA

◆ SECTION: DATA

```
#****Begin: Generated Statements***
#****End: Generated Statements****

nRecordCount = nRecordCount + 1;

# ═══════════════════════════════════════════════════════════════════════
# DEPTH-AWARE EXPANSION
# Center : ELLEV 0=leaf, 1=mid-parent (Corp Centers), 2=top-parent
# Location: ELLEV 0=leaf, 1=parent (Location Control Total)
# Uses ELCOMPN/ELCOMP only — no ELISANC, no full dim scan
# ═══════════════════════════════════════════════════════════════════════

IF( ELLEV( 'Center', dCenter ) = 0 );

    #── CENTER IS LEAF ──────────────────────────────────────────────
    IF( ELLEV( 'Location', dLocation ) = 0 );

        # Scenario 1: Center leaf x Location leaf
        IF( CELLGETS( vTargetCube, dUser, dCenter, dLocation, vTargetMeasure ) @<> 'WRITE' );
            CELLPUTS( dValue, vTargetCube, dUser, dCenter, dLocation, vTargetMeasure );
            nRecordProcess = nRecordProcess + 1;
        ELSE;
            nRecordSkip = nRecordSkip + 1;
        ENDIF;

    ELSE;

        # Scenario 2: Center leaf x Location parent (ELLEV=1)
        vLocCount = ELCOMPN( 'Location', dLocation );
        vLocIdx   = 1;
        WHILE( vLocIdx <= vLocCount );
            vLocLeaf = ELCOMP( 'Location', dLocation, vLocIdx );
            IF( CELLGETS( vTargetCube, dUser, dCenter, vLocLeaf, vTargetMeasure ) @<> 'WRITE' );
                CELLPUTS( dValue, vTargetCube, dUser, dCenter, vLocLeaf, vTargetMeasure );
                nRecordProcess = nRecordProcess + 1;
            ELSE;
                nRecordSkip = nRecordSkip + 1;
            ENDIF;
            vLocIdx = vLocIdx + 1;
        END;

    ENDIF;

ELSEIF( ELLEV( 'Center', dCenter ) = 1 );

    #── CENTER IS MID-PARENT (ELLEV=1) — direct children are leaves ───
    vCenCount = ELCOMPN( 'Center', dCenter );
    vCenIdx   = 1;
    WHILE( vCenIdx <= vCenCount );
        vCenLeaf = ELCOMP( 'Center', dCenter, vCenIdx );

        IF( ELLEV( 'Location', dLocation ) = 0 );

            # Scenario 3: Center mid-leaf x Location leaf
            IF( CELLGETS( vTargetCube, dUser, vCenLeaf, dLocation, vTargetMeasure ) @<> 'WRITE' );
                CELLPUTS( dValue, vTargetCube, dUser, vCenLeaf, dLocation, vTargetMeasure );
                nRecordProcess = nRecordProcess + 1;
            ELSE;
                nRecordSkip = nRecordSkip + 1;
            ENDIF;

        ELSE;

            # Scenario 4: Center mid-leaf x Location parent (ELLEV=1)
            vLocCount = ELCOMPN( 'Location', dLocation );
            vLocIdx   = 1;
            WHILE( vLocIdx <= vLocCount );
                vLocLeaf = ELCOMP( 'Location', dLocation, vLocIdx );
                IF( CELLGETS( vTargetCube, dUser, vCenLeaf, vLocLeaf, vTargetMeasure ) @<> 'WRITE'
);
```

```
                    CELLPUTS( dValue, vTargetCube, dUser, vCenLeaf, vLocLeaf, vTargetMeasure );
                    nRecordProcess = nRecordProcess + 1;
                ELSE;
                    nRecordSkip = nRecordSkip + 1;
                ENDIF;
                vLocIdx = vLocIdx + 1;
            END;

        ENDIF;

        vCenIdx = vCenIdx + 1;
    END;

ELSE;

    #── CENTER IS TOP-PARENT (ELLEV=2) — 2 levels to reach leaves ──────
    # THIS WAS THE BUG: old code stopped at Corp Centers (ELLEV=1)
    # and never reached the actual leaf centers
    vCenCount1 = ELCOMPN( 'Center', dCenter );
    vCenIdx1   = 1;
    WHILE( vCenIdx1 <= vCenCount1 );
        vCenMid = ELCOMP( 'Center', dCenter, vCenIdx1 );   # ELLEV=1

        vCenCount2 = ELCOMPN( 'Center', vCenMid );
        vCenIdx2   = 1;
        WHILE( vCenIdx2 <= vCenCount2 );
            vCenLeaf = ELCOMP( 'Center', vCenMid, vCenIdx2 );   # ELLEV=0

            IF( ELLEV( 'Location', dLocation ) = 0 );

                # Scenario 5: Center top-leaf x Location leaf
                IF( CELLGETS( vTargetCube, dUser, vCenLeaf, dLocation, vTargetMeasure ) @<>
'WRITE' );
                    CELLPUTS( dValue, vTargetCube, dUser, vCenLeaf, dLocation, vTargetMeasure );
                    nRecordProcess = nRecordProcess + 1;
                ELSE;
                    nRecordSkip = nRecordSkip + 1;
                ENDIF;

            ELSE;

                # Scenario 6: Center top-leaf x Location parent (ELLEV=1)
                vLocCount = ELCOMPN( 'Location', dLocation );
                vLocIdx   = 1;
                WHILE( vLocIdx <= vLocCount );
                    vLocLeaf = ELCOMP( 'Location', dLocation, vLocIdx );
                    IF( CELLGETS( vTargetCube, dUser, vCenLeaf, vLocLeaf, vTargetMeasure ) @<>
'WRITE' );
                        CELLPUTS( dValue, vTargetCube, dUser, vCenLeaf, vLocLeaf, vTargetMeasure
);
                        nRecordProcess = nRecordProcess + 1;
                    ELSE;
                        nRecordSkip = nRecordSkip + 1;
                    ENDIF;
                    vLocIdx = vLocIdx + 1;
                END;

            ENDIF;

            vCenIdx2 = vCenIdx2 + 1;
        END;

        vCenIdx1 = vCenIdx1 + 1;
    END;

ENDIF;
```

7. EPILOG

◆ SECTION: EPILOG

```
#****Begin: Generated Statements***
#****End: Generated Statements****

# Turn On Cube Logging
CELLPUTS( 'YES', '}CubeProperties', vTargetCube, 'LOGGING' );

# ── Destroy ZeroOut View and Subsets ──────────────────────────────
IF( VIEWEXISTS( vTargetCube, vZeroOutView ) = 1 );
    VIEWDESTROY( vTargetCube, vZeroOutView );
ENDIF;
IF( SUBSETEXISTS( 'Center Location Security Measure', vZeroOutSubset ) = 1 );
    SUBSETDESTROY( 'Center Location Security Measure', vZeroOutSubset );
ENDIF;

# ── Destroy Source View and Subsets ───────────────────────────────
IF( VIEWEXISTS( vSourceCube, vSourceView ) = 1 );
    VIEWDESTROY( vSourceCube, vSourceView );
ENDIF;
IF( SUBSETEXISTS( '}Clients', vSourceSubset ) = 1 );
    SUBSETDESTROY( '}Clients', vSourceSubset );
ENDIF;
IF( SUBSETEXISTS( 'Center', vSourceSubset ) = 1 );
    SUBSETDESTROY( 'Center', vSourceSubset );
ENDIF;
IF( SUBSETEXISTS( 'Location', vSourceSubset ) = 1 );
    SUBSETDESTROY( 'Location', vSourceSubset );
ENDIF;
IF( SUBSETEXISTS( 'Center Location Security Measure', vSourceSubset ) = 1 );
    SUBSETDESTROY( 'Center Location Security Measure', vSourceSubset );
ENDIF;

# ── Final Process Stats ───────────────────────────────────────────
sLogFile = GETPROCESSERRORFILEDIRECTORY | GETPROCESSERRORFILENAME;
sNow     = TIMST( NOW, '\M \D, \Y at \H \p \i min \s sec' );
vNowEnd  = NOW;

vDelta    = ( vNowEnd - vNowStart );
vDuration = vDelta * 1440;
vDurMin   = INT( vDuration );
IF( vDurMin = 0 );
    vDurSec = ROUND( vDuration * 60 );
ELSE;
    vDurSec = ROUND( MOD( vDuration, vDurMin ) * 60 );
ENDIF;
sDuration = NUMBERTOSTRING( vDurMin ) | ' min ' | NUMBERTOSTRING( vDurSec ) | ' sec';

CELLPUTS( sDuration, sCube, CurrLine, sDate,   sProcess, 'Duration' );
CELLPUTS( sDuration, sCube, CurrLine, 'Latest', sProcess, 'Duration' );
CELLPUTS( sNow,      sCube, CurrLine, sDate,   sProcess, 'End Time' );
CELLPUTS( sNow,      sCube, CurrLine, 'Latest', sProcess, 'End Time' );

CELLPUTN( nRecordCount,   sCube, CurrLine, sDate,   sProcess, 'Record Count' );
CELLPUTN( nRecordCount,   sCube, CurrLine, 'Latest', sProcess, 'Record Count' );
CELLPUTN( nRecordSkip,    sCube, CurrLine, sDate,   sProcess, 'Skipped Records' );
CELLPUTN( nRecordSkip,    sCube, CurrLine, 'Latest', sProcess, 'Skipped Records' );

nRecordProcess = nRecordCount - nRecordSkip;
CELLPUTN( nRecordProcess, sCube, CurrLine, sDate,   sProcess, 'Processed Records' );
CELLPUTN( nRecordProcess, sCube, CurrLine, 'Latest', sProcess, 'Processed Records' );
```